

数值分析期末上机报告

学号：241098117 姓名：张城宗

1 题目一：经典四阶 Runge-Kutta 方法

1.1 问题描述

求解初值问题

$$y' = f(x, y), \quad x \in [x_0, X], \quad y(x_0) = y_0,$$

取步长 $h =$ _____, 使用 _____ 方法计算数值解, 并分析误差。

1.2 方法与公式

以经典四阶 Runge-Kutta 方法为例:

$$\begin{aligned} K_1 &= f(x_n, y_n), \\ K_2 &= f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_1\right), \\ K_3 &= f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}K_2\right), \\ K_4 &= f(x_n + h, y_n + hK_3), \\ y_{n+1} &= y_n + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4). \end{aligned}$$

若题目给出精确解 $y(x)$, 使用最大绝对误差

$$E_\infty = \max_n |y_n - y(x_n)|$$

评价数值结果。若需验证收敛阶, 则使用

$$p \approx \frac{\log(E(h_1)/E(h_2))}{\log(h_1/h_2)}.$$

1.3 Python 实现说明

程序首先建立等距网格, 再按上述公式逐步推进; 最后输出节点结果、最大绝对误差和终点误差, 并生成解曲线与误差曲线。

1.4 数值结果

表 1: 各节点数值结果

x_n	数值解 y_n	精确解 $y(x_n)$	绝对误差
0.000000	1.0000000000	1.0000000000	0.000e+00
0.100000	1.0003252083	1.0003251639	4.441e-08
0.200000	1.0025385940	1.0025384938	1.002e-07
0.300000	1.0083637234	1.0083635586	1.648e-07
0.400000	1.0193601439	1.0193599079	2.360e-07
0.500000	1.0369389926	1.0369386806	3.120e-07
0.600000	1.0623771190	1.0623767278	3.912e-07
0.700000	1.0968298648	1.0968293924	4.723e-07
0.800000	1.1413426261	1.1413420718	5.543e-07
0.900000	1.1968613168	1.1968606805	6.362e-07
1.000000	1.2642418350	1.2642411177	7.174e-07

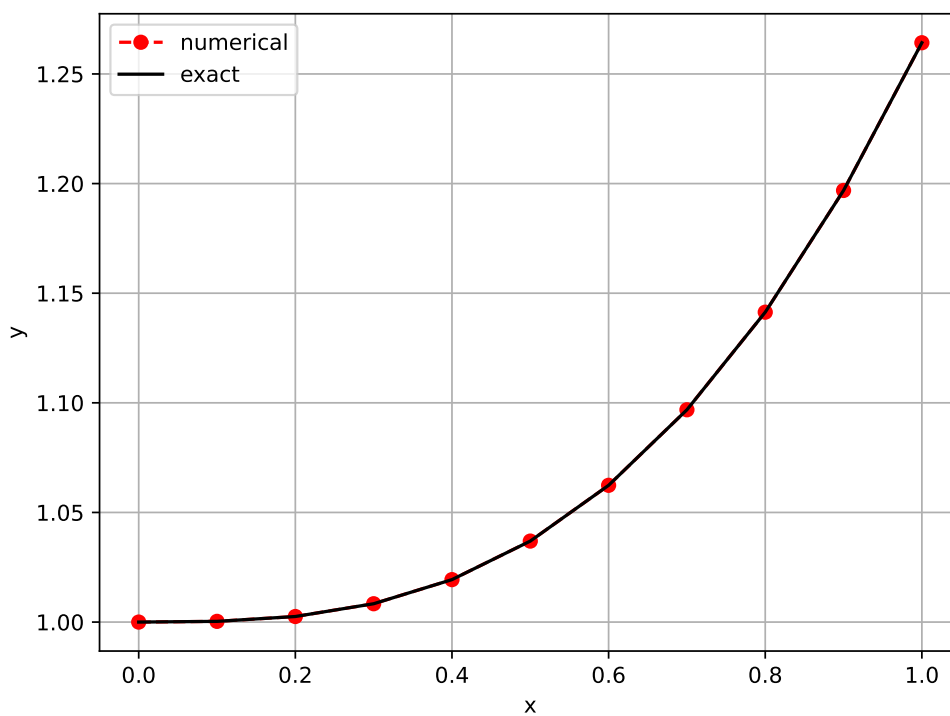


图 1: 数值解与精确解比较

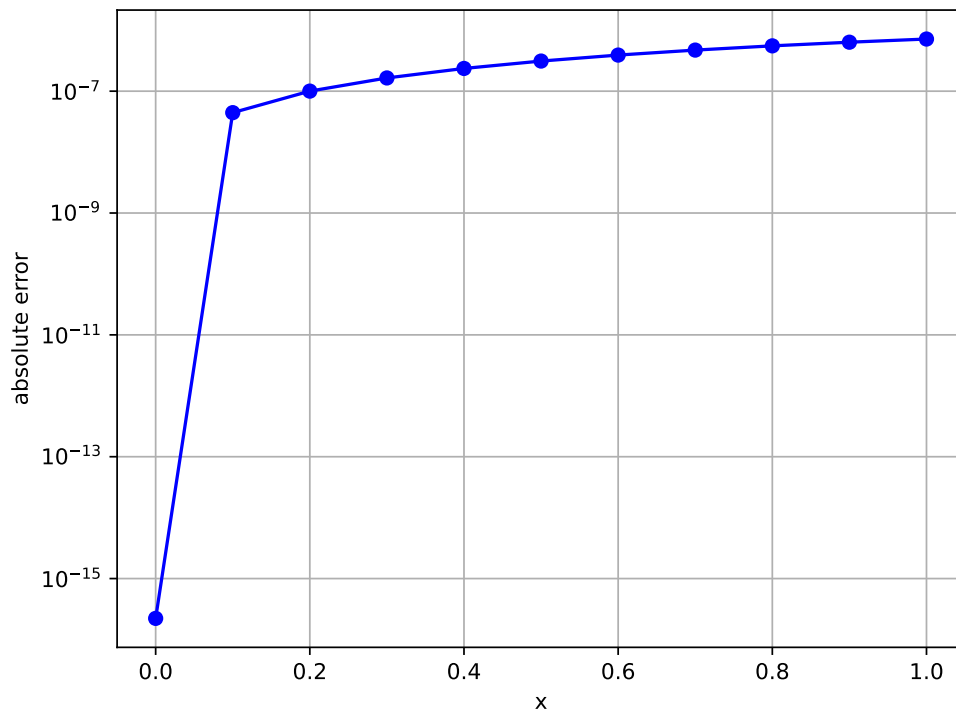


图 2: 各节点绝对误差

1.5 结果分析

1. 数值现象:【必须替换】说明数值解的变化趋势, 以及是否符合题意。
2. 误差与精度:【必须替换】报告最大误差、终点误差或估计收敛阶。
3. 原因解释:【必须替换】联系方法阶数、步长、稳定性或迭代容差解释结果。

1.6 结论

【必须替换】用两三句话回答题目要求, 并给出最重要的数值结论。

附录: Python 程序

Listing 1: 本题 Python 程序

```
1  """单道数值 ODE 上机题模板: 修改“必须替换”区域后直接运行。"""
2
3  from pathlib import Path
4
5  import matplotlib.pyplot as plt
6  import numpy as np
7
8  OUTPUT_DIR = Path(__file__).resolve().parent
9
10 # ===== 必须替换 =====
11 f = lambda x, y: -y + x**2 + 1 # 方程右端 f(x, y)。
12 exact = lambda x: x**2 - 2 * x + 3 - 2 * np.exp(-x) # 无精确解时改为 None。
13 x0, x_end, y0, h = 0.0, 1.0, 1.0, 0.1 # 区间、初值、步长。
14 # =====
15
16
17 def rk4(f, x0, x_end, y0, h):
18     """经典四阶 Runge--Kutta; 标量与 NumPy 向量初值均可使用。"""
19     n_steps = round((x_end - x0) / h)
20     if not np.isclose(x0 + n_steps * h, x_end):
21         raise ValueError("步长 h 必须整除积分区间长度。")
22
23     x = x0 + np.arange(n_steps + 1) * h
24     y0_array = np.asarray(y0, dtype=float)
25     y = np.zeros((n_steps + 1,) + y0_array.shape, dtype=float)
26     y[0] = y0_array
27
28     for n in range(n_steps):
29         k1 = f(x[n], y[n])
30         k2 = f(x[n] + h / 2, y[n] + h * k1 / 2)
31         k3 = f(x[n] + h / 2, y[n] + h * k2 / 2)
32         k4 = f(x[n] + h, y[n] + h * k3)
33         y[n + 1] = y[n] + h * (k1 + 2 * k2 + 2 * k3 + k4) / 6
34     return x, y
35
36
37 def write_latex_table(x, y, y_exact=None):
38     """生成可由 LaTeX 主文件直接 input 的三线表。"""
39     lines = [
40         r"\begin{table}[H]",
41         r" \centering",
42         r" \caption{各节点数值结果}",
43     ]
44     if y_exact is None:
45         lines += [
46             r" \begin{tabular}{cc}",
47             r" \toprule",
```

```

48         r"      $x_n$ & 数值解 $y_n$ \\",
49         r"      \midrule",
50     ]
51     lines += [f"      {xn:.6f} & {yn:.10f} \\\\" for xn, yn in zip(x, y)]
52 else:
53     lines += [
54         r"      \begin{tabular}{cccc}",
55         r"      \toprule",
56         r"      $x_n$ & 数值解 $y_n$ & 精确解 $y(x_n)$ & 绝对误差 \\",
57         r"      \midrule",
58     ]
59     lines += [
60         f"      {xn:.6f} & {yn:.10f} & {ye:.10f} & {abs(yn - ye):.3e} \\\\"
61         for xn, yn, ye in zip(x, y, y_exact)
62     ]
63     lines += [r"      \bottomrule", r"      \end{tabular}", r"\end{table}"]
64     (OUTPUT_DIR / "result_table.tex").write_text("\n".join(lines), encoding="utf-8")
65
66
67 def save_figures(x, y, y_exact=None):
68     """生成供 LaTeX 自动插入的 PDF 图片。"""
69     plt.plot(x, y, "ro--", label="numerical")
70     if y_exact is not None:
71         plt.plot(x, y_exact, "k-", label="exact")
72     plt.xlabel("x")
73     plt.ylabel("y")
74     plt.grid(True)
75     plt.legend()
76     plt.tight_layout()
77     plt.savefig(OUTPUT_DIR / "solution.pdf")
78     plt.close()
79
80     if y_exact is not None:
81         plt.semilogy(x, np.maximum(np.abs(y - y_exact), np.finfo(float).eps), "bo-")
82         plt.xlabel("x")
83         plt.ylabel("absolute error")
84         plt.grid(True, which="both")
85         plt.tight_layout()
86         plt.savefig(OUTPUT_DIR / "error.pdf")
87         plt.close()
88
89
90 def main():
91     x, y = rk4(f, x0, x_end, y0, h)
92     if y.ndim != 1:
93         raise ValueError("当前结果表按标量方程编写; 方程组请选取要报告的分量。")
94
95     y_exact = None if exact is None else exact(x)
96     write_latex_table(x, y, y_exact)
97     save_figures(x, y, y_exact)
98

```

```
99     print("x          numerical")
100     for xn, yn in zip(x, y):
101         print(f"{xn:8.4f}      {yn:.10f}")
102     if y_exact is not None:
103         errors = np.abs(y - y_exact)
104         print(f"maximum absolute error = {errors.max():.6e}")
105         print(f"endpoint absolute error = {errors[-1]:.6e}")
106
107
108 if __name__ == "__main__":
109     main()
```